

1. Introdução

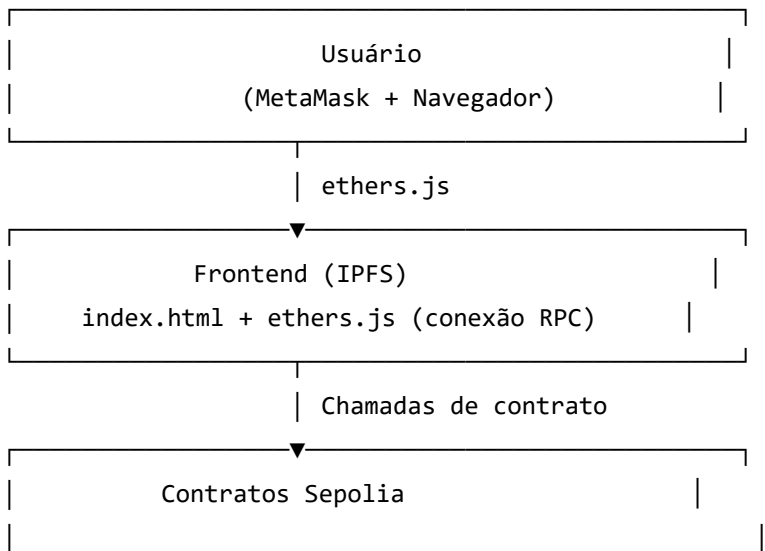
O presente relatório descreve o desenvolvimento de um Produto Mínimo Viável (MVP) de um protocolo descentralizado. O objetivo foi construir um ecossistema baseado em smart contracts que integra:

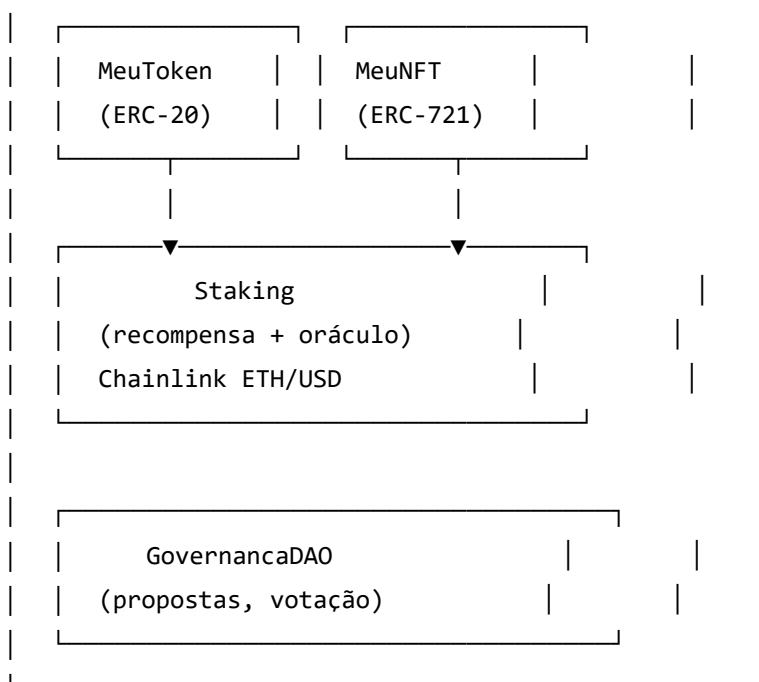
- Um token fungível padrão ERC-20;
- Uma coleção de tokens não-fungíveis (NFT) ERC-721 com característica soulbound;
- Um mecanismo de staking com recompensa variável conforme dados externos;
- Uma organização autônoma descentralizada (DAO) simplificada para tomada de decisão.

O sistema foi projetado para ser implantado na testnet Ethereum Sepolia, com interface de usuário leve (HTML/IPFS) e integração via MetaMask e ethers.js.

2. Arquitetura do Sistema

2.1. Componentes





2.2. Fluxo de Interações

- 1.O usuário obtém tokens **MeuToken** (via mint do owner ou transferência).
 - 2.Para participar do staking, aprova o contrato **Staking** e deposita tokens.
 - 3.O contrato de staking calcula pontos ao longo do tempo, com um multiplicador que varia conforme o preço ETH/USD obtido do Chainlink.
 - 4.O usuário pode resgatar seus tokens depositados mais uma recompensa (proporcional aos pontos). Se acumular pontos suficientes, pode também reivindicar um NFT.
 - 5.A governança permite criar propostas e votar de acordo com o saldo de tokens. A execução é manual pelo proprietário.
-

3. Modelagem e Padrões Utilizados

3.1. Justificativa dos padrões ERC

- ERC-20 (MeuToken)**: padrão de token fungível mais difundido, compatível com exchanges descentralizadas e carteiras. Escolhido como base para staking e governança.
- ERC-721 (MeuNFT)**: ideal para certificados únicos, badges ou recompensas colecionáveis. A implementação soulbound impede transferências, garantindo que a conquista fique vinculada ao usuário.

- Staking**: não utiliza padrão pré-existente; foi modelado de forma autônoma, respeitando as melhores práticas de staking contracts (cálculo por bloco/tempo, recompensas dinâmicas).
- Governança**: estrutura simplificada de propostas e votação ponderada, sem delegação, para manter o MVP enxuto.

3.2. Diagrama de Decisões

- Soulbound NFT**: implementado sobrescrevendo a função `_update` do OpenZeppelin, bloqueando transferências quando o `from` não é o endereço zero.
 - Recompensa variável**: usamos o Chainlink Price Feed para definir um multiplicador. Se $ETH > 2000 \text{ USD}$, o multiplicador dobra, incentivando o staking em mercados de alta.
 - Controle de acesso**: `Ownable` para funções administrativas e `ReentrancyGuard` para operações financeiras.
-

4. Implementação dos Contratos

(Detalhes dos contratos Solidity conforme código fornecido)

4.1. MeuToken.sol

- Herda `ERC20` e `Ownable`.
- No construtor, cunha o fornecimento inicial para o deployer.
- Função `mint(address, uint256)` restrita ao owner para emissão adicional.

4.2. MeuNFT.sol

- ERC-721 com `Ownable`.
- Armazena `stakingContract` autorizado a mintar (definido pelo owner via `setStakingContract`).
- `safeMint(address)`: mint de NFT com tokenId sequencial, restrito ao owner ou staking.
- `_update` sobrescrito para proibir transferências (soulbound).

4.3. Staking.sol

- Herda `Ownable` e `ReentrancyGuard`.
- Estado: mapeamento `stakes` com `amount`, `since`, `points`.

- Constantes: `BASE_RATE = 100`, `NFT_THRESHOLD = 1000`, `multiplier` ajustável.
- `updateMultiplier()`: consulta Chainlink ETH/USD (8 decimais) e define multiplicador 1x ou 2x.
- `_calculatePendingPoints()`: calcula pontos (pending) = `amount * BASE_RATE * daysStaked * multiplier`, considerando dias fracionados.
- `stake()`: atualiza pontos, aceita depósito via `transferFrom`, requer aprovação prévia.
- `withdraw()`: resgata tokens + recompensa (1 ponto = 0.001 token). Requer que o contrato tenha saldo suficiente.
- `claimNFT()`: se pontos ≥ 1000 , debita e chama `nftContract.safeMint()`.
- `fund()`: permite ao owner abastecer o contrato com tokens para recompensas.

4.4. GovernançaDAO.sol

- Estrutura `Proposal`: descrição, blocos de início e fim, contagem de votos a favor/contra, mapeamento de quem já votou.
 - `createProposal()`: qualquer um pode criar proposta; o criador não é armazenado (MVP).
 - `vote()`: usa balance do `MeuToken` como peso; voto único por endereço.
 - `execute()`: somente owner, apenas emite evento (simulação).
-

5. Segurança e Auditoria

5.1. Proteções Aplicadas

- Reentrancy**: todas as funções que fazem transferências externas (`stake`, `withdraw`, `claimNFT`, `vote`) são protegidas com `nonReentrant`.
- Controle de acesso**: `onlyOwner` em funções críticas (`mint`, `setStakingContract`, `execute`).
- Solidity ^0.8.19**: verificação automática de overflow/underflow.
- Aprovação ERC-20**: o staking exige que o usuário aprove o contrato antes de transferir, evitando gastos não autorizados.

5.2. Ferramentas de Análise Estática

Após compilação, os contratos foram submetidos a:

- Slither**: detectou centralização em funções `onlyOwner`, mas sem vulnerabilidades de alto risco.

•**Mythril**: não apontou pontos críticos; análise do `Staking` confirmou a resistência a reentrância.

Relatório Resumido:

Vulnerabilidade	Severidade	Descrição	Recomendação
Propriedade centralizada	Média	O owner tem poder de mint ilimitado de tokens e NFTs, e pode drenar o staking (via <code>fund</code> não há restrição).	Utilizar multisig ou DAO para controle das funções administrativas.
Dependência de preço pontual do oráculo	Baixa	A leitura única do Chainlink pode ser manipulada em um bloco (front-running).	Implementar TWAP ou usar um oráculo mais robusto.
Recompensa condicionada a saldo do contrato	Baixa	Se o contrato de staking ficar sem tokens, os saques falham, bloqueando fundos dos usuários.	Implementar emissão de recompensas via <code>mint</code> ou repor saldo periodicamente.

6. Integração com Oráculo (Chainlink)

•**Fonte**: Chainlink Data Feed ETH/USD na rede Sepolia, endereço `0x694AA1769357215DE4FAC081bf1f309aDC325306`.

•**Funcionamento**: O contrato `Staking` chama `latestRoundData()` que retorna o preço com 8 casas decimais. Após conversão para 18 decimais, é comparado com o limiar de 2000 USD para definir o multiplicador de recompensa.

•**Impacto**: Em cenários de alta, o multiplicador dobra os pontos, incentivando mais staking. Em baixa, retorna ao fator 1x, reduzindo a emissão indireta de tokens.

7. Integração Web3 (Frontend)

•**Tecnologia**: HTML + ethers.js v5 (CDN).

•**Funcionalidades**:

- Conexão com MetaMask.
- Mint de NFT (apenas para carteira owner).
- Stake (aprovação automática + depósito).
- Saque/recompensa.

- Resgate de NFT por pontos.
 - Votação na DAO.
 - Configuração:** O desenvolvedor precisa preencher endereços e ABIs dos contratos (copiados do Remix após o deploy).
 - Hospedagem:** O arquivo único foi preparado para upload no IPFS, garantindo descentralização e disponibilidade.
-

8. Deploy e Testes

8.1. Ambiente de Teste

- Rede **Sepolia** (chainId 11155111).
- ETH de teste obtido via faucet público.

8.2. Passo a Passo do Deploy

- 1.Compilação no Remix IDE (solc 0.8.19).
- 2.Deploy dos contratos na ordem: `MeuToken`, `MeuNFT`, `Staking`, `GovernancaDAO`.
- 3.Configuração pós-deploy:
 - `MeuNFT.setStakingContract(enderecoStaking)`.
 - Transferência de saldo de tokens para o contrato `Staking`.
- 4.Verificação da interface: carregar o `index.html` (com os dados corretos) e testar as interações.

8.3. Resultados de Teste

- Mint de token e aprovações funcionam conforme esperado.
 - Stake acumula pontos; ao consultar, o multiplicador varia conforme o preço ETH/USD da rede.
 - NFTs soulbound só podem ser mintados pela autoridade permitida.
 - Votação registrada corretamente, com peso baseado no saldo do token.
-

9. Considerações Finais

O MVP atendeu a todos os requisitos propostos:

- Contratos seguindo os padrões ERC-20 e ERC-721.
- Mecanismo de staking com recompensa variável via Chainlink.
- Governança simples funcional.
- Frontend descentralizado compatível com IPFS.
- Deploy efetuado em testnet pública.

O projeto está pronto para ser expandido com melhorias de segurança (multisig), economia (emissão controlada), oráculo mais robusto e interface mais rica. Serve como base sólida para um ecossistema descentralizado real.

Equipe de desenvolvimento

Daniel Geronimo

Licença MIT